

The Big-Picture Map: How the Stages of the pyMack Workflow Fit Together

What each stage consumes, what it produces, which formulation and solver is used when, and how the chain ends at a transition estimate

pyMack technical documentation

July 3, 2026

Contents

1	Scope	1
2	The pipeline at a glance	2
3	Stage 1–2: the mean boundary layer and its consumers	3
4	Stage 3, first decision: temporal or spatial formulation	4
5	Stage 3, second decision: which eigenvalue engine	5
6	Stages 5–6: from eigenvalues to deliverables	6
7	The canonical Mach 6 case, stage by stage	7
8	Where validation attaches to the map	8

1 Scope

The five companion documents each descend into one stage of the linear stability workflow: the steady base flow (*Mean Boundary Layer*), the linearised operator and its discretisation (*Derivation of the Disturbance Equations*), the spectral eigenvalue algorithms (*Numerical Methods*), the independent local solver (*Shooting Solver*), and the evidence (*Validation*). What none of them states in one place is the connective tissue: at which stage the boundary-layer profile is actually consumed, which of the two stability formulations (temporal or spatial) is used for which product, when the global eigenvalue solver is preferred over the local shooting solver, and how the chain of intermediate objects terminates in a neutral curve and an N -factor. This document supplies that map. Section 2 presents the pipeline end to end; Section 3 traces every consumer of the mean flow; Section 4 states the division of labour between the temporal and spatial formulations; Section 5 does the same for the four eigenvalue engines; Section 6 follows the eigenvalues into neutral curves, N -factors, and dimensional predictions; Section 7 walks the canonical Mach 6 production case through every stage; Section 8 locates the validation layers on the map. Nothing here is new

material: every statement is a cross-reference into the companion documents and the code, cited by file.

2 The pipeline at a glance

Everything the framework produces is assembled from one repeated primitive: an eigenvalue solve of the linearised disturbance equations at a single parameter point. The pipeline is therefore a chain of *preparations* for that solve (stages 1–2), the *solve itself* in one of two formulations by one of several engines (stage 3), an *acceptance test* that separates the physical mode from discretisation artifacts (stage 4), and *aggregation* of many accepted solves into the engineering products (stages 5–6). For a normal-mode disturbance [1, 5]

$$q'(x, y, z, t) = \hat{q}(y) e^{i(\alpha x + \beta z - \omega t)}, \quad (1)$$

the two formulations differ only in which of (α, ω) is prescribed real and which is the complex unknown:

$$\text{temporal: given } \alpha \in \mathbb{R}, \text{ find } c = \omega/\alpha \in \mathbb{C} (\omega_i = \alpha c_i); \quad \text{spatial: given } \omega \in \mathbb{R}, \text{ find } \alpha \in \mathbb{C} (\sigma = -\alpha_i). \quad (2)$$

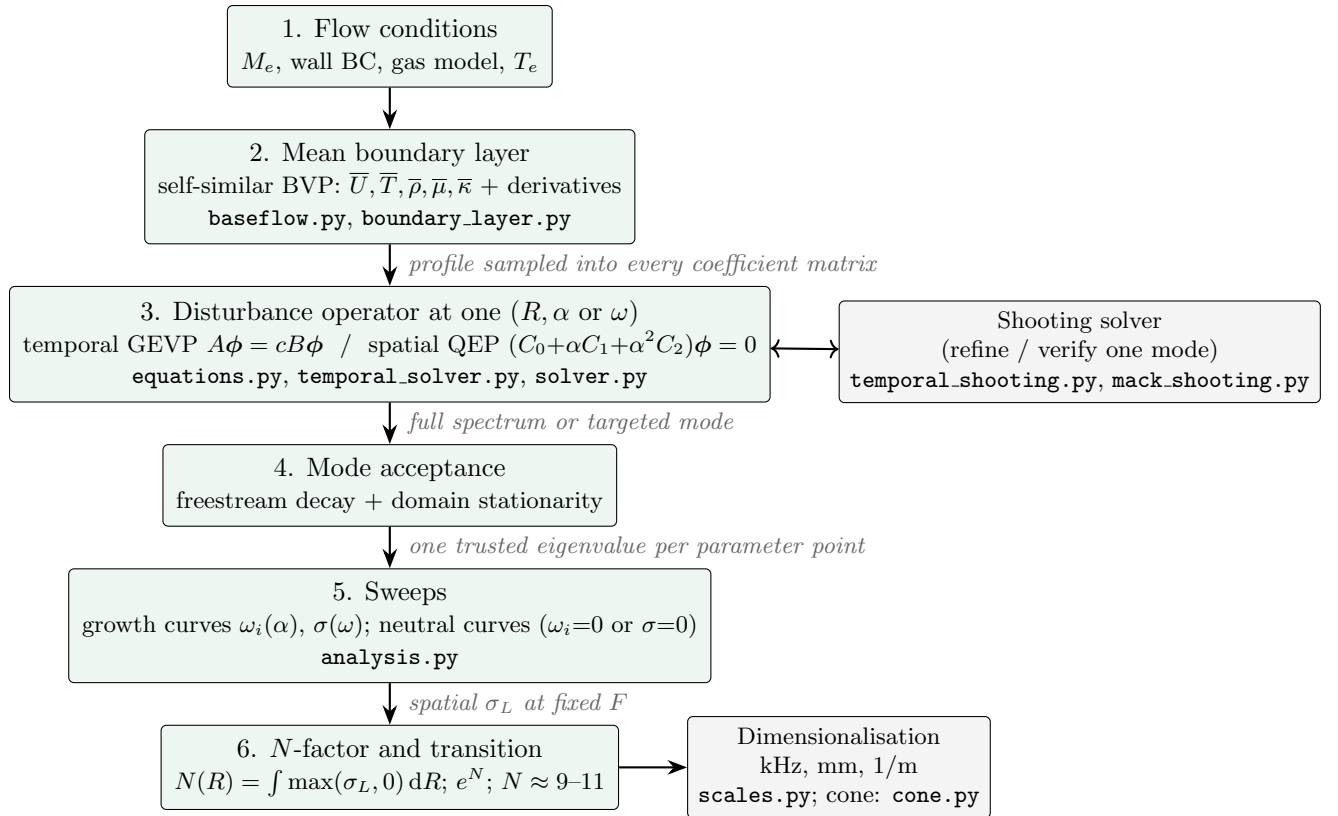


Figure 1: The pyMack pipeline. The mean flow (stage 2) is computed once per flow condition and consumed by every operator assembly (stage 3); stages 3–4 constitute the repeated eigenvalue primitive; stages 5–6 aggregate thousands of such solves into the engineering products. The shooting solver sits beside stage 3 as an independent second route to the same eigenvalue.

Table 1 states, for each stage, what enters, what leaves, and who consumes the output. The remainder of the document expands each row.

Stage	Input	Output	Consumed by
Mean flow	M_e , wall BC, transport law	profiles $\bar{U}, \bar{T}, \bar{\rho}, \bar{\mu}, \bar{\kappa}$ and y -derivatives on a similarity grid; δ^*/L^*	every coefficient matrix of stage 3; scale conversions; $y_{\max}$ sizing
Operator assembly	profiles sampled on the collocation grid; (R, α) or (R, ω)	matrix pencil (A, B) or QEP blocks (C_0, C_1, C_2) ; pointwise 6×6 $A(y)$ for shooting	eigenvalue engines
Eigenvalue solve	the operator + (spatial only) a shift target or seed	raw spectrum, or one refined eigenvalue	mode acceptance
Mode acceptance	raw eigenvalues + eigenfunctions	the discrete boundary-layer mode	growth curves, neutral tracing
Sweeps	accepted modes over (R, α) or (R, ω) grids	ω_i or σ_L fields; neutral branches	N -factor; overlays; verification verdicts
N -factor	$\sigma_L(R)$ at fixed F	$N(R)$, $A/A_0 = e^N$	transition estimate; dimensional plots

Table 1: Stage-by-stage data flow. One mean-flow solve serves an entire stability survey; one eigenvalue solve serves exactly one parameter point.

3 Stage 1–2: the mean boundary layer and its consumers

The base flow answers a question that must be settled before any stability problem exists: about *what* state are the equations linearised? The self-similar compressible flat-plate solution (*Mean Boundary Layer*, Sections 2–3; `pymack/baseflow.py`) provides $\bar{U}(y), \bar{T}(y), \bar{\rho} = 1/\bar{T}$, the transport properties $\bar{\mu}(\bar{T}), \bar{\kappa}(\bar{T})$ with their temperature derivatives, and the first and second y -derivatives of everything, on a wall-normal grid.

The profile is computed once and reused everywhere. Because the solution is self-similar, the nondimensional profile depends only on $(M_e, \text{wall condition, transport law, } T_e)$ and *not* on the streamwise station: the Reynolds number R enters the disturbance operator as an explicit parameter (through the viscous prefactor $\nu_R = 1/(Re\bar{\rho})$), not through the profile. One call to `flat_plate/make_mack_profile/generate_boundary_layer` therefore serves an entire (R, α) or (R, ω) survey of thousands of eigenvalue solves. This is the single most economical fact of the workflow: stage 2 costs one boundary-value solve [14], stage 3–4 cost one dense eigensolve *per parameter point*.

Consumer 1: the coefficient matrices. The profile’s role in the eigenvalue problem is to populate coefficients. Every entry of the temporal blocks A, B (`temporal_solver.py`), of the spatial blocks C_0, C_1, C_2 (`equations.py`), and of the pointwise shooting matrix $A(y)$ (`mack_shooting.py`) is an algebraic combination of $\bar{U}, \bar{U}', \bar{U}'', \bar{T}, \bar{T}', \bar{T}'', \bar{\rho}, \bar{\mu}, d\mu/dT, d^2\mu/dT^2, \bar{\kappa}$ and its derivatives, evaluated at the grid points (*Numerical Methods*, Section 2; *Shooting Solver*, Section 2). The profile is sampled onto the Chebyshev collocation grid (or the RK4 marching grid) by the single shared resampler `scales.sample_baseflow`; the solvers never re-solve the mean flow.

Consumer 2: the length scales. The stability solvers store profiles on a y/δ^* grid, while Mack’s figures and all neutral-curve axes use the Falkner–Skan scale $L^* = \sqrt{\nu_e x/U_e}$ with $R = \sqrt{Re_x}$ [1]. The conversion factor δ^*/L^* is a property of the profile, computed by quadrature of $(\bar{\rho}\bar{U})$ deficit across the layer, and every $\alpha_{\delta^*} \leftrightarrow \alpha_L$ or $y_{\delta^*} \leftrightarrow y_L$ conversion in the repository routes through `pymack/scales.py` (*Mean Boundary Layer*, Section 4).

Consumer 3: the domain height. The collocation domain is truncated at $y_{\max} = m(\delta^*/L^*)$ with $m \approx 8$ –12 for the wall-trapped second mode and $m \approx 35$ –45 for the diffuse first mode (*Numerical Methods*, Section 2), so even the grid geometry of stage 3 is set by a boundary-layer thickness that stage 2 supplies.

What the profile is *not* used for. The N -factor integration and the dimensionalisation consume only eigenvalue outputs (σ_L vs R) and an edge state (U_e, ν_e, T_e); the profile does not reappear downstream of stage 3. If two runs share a profile and differ in R , their difference is carried entirely by the operator, not by the base flow.

4 Stage 3, first decision: temporal or spatial formulation

The repository genuinely uses *both* formulations of Eq. (2), not as redundant alternatives but with a clean division of labour that follows from what each returns and what each costs.

Why the temporal problem is the survey tool. With α real and c the eigenvalue, the discretised problem is the *linear* generalised eigenvalue problem

$$\boxed{A(\alpha)\phi = cB(\alpha)\phi} \quad (3)$$

(*Numerical Methods*, Section 3), solved by QZ [6] for *all* $4n$ eigenvalues at once with no initial guess. This no-guess property is the reason the temporal formulation is the discovery and survey tool [2]: when nothing is yet known about the spectrum at a new (M_e, R) , a temporal solve finds the second mode (phase speed $c_r \approx 1 - 1/M_e$), the first mode ($c_r \approx 0.3$ –0.6), and the continuous-spectrum clusters simultaneously. Temporal sweeps produce the growth maps $\omega_i(R, \alpha)$, the temporal neutral curves, and the validation comparisons against Mack’s temporal tables (Table 10.1 growth rates) and Özgen & Kırca’s Fig. 3 lobes [1, 3].

Why the spatial problem is the transition tool. A physical disturbance source (roughness, acoustic forcing) oscillates at a fixed real frequency and grows in *space*, so transition prediction needs $\sigma = -\alpha_i$ at prescribed real ω ; the N -factor integral is a streamwise integral of spatial growth and cannot be built from temporal quantities without an approximate group-velocity conversion [8]. With α as the eigenvalue the viscous terms make the problem *quadratic*,

$$(C_0 + \alpha C_1 + \alpha^2 C_2)\phi = 0, \quad (4)$$

linearised to companion form at twice the size and solved near a shift target $\alpha_0 = \omega/c_{\text{ph}}$ [7] (*Numerical Methods*, Section 4). The spatial formulation therefore requires an idea of where the mode lies; it is used once the mode family is known, to produce the quantities transition actually consumes: fixed-frequency growth curves $\sigma_L(R)$, spatial neutral branches, and N -factors.

The bridge between the two. The two formulations meet in `solve_spatial_from_temporal` (`solver.py`): a temporal solve at $\alpha_r = \omega/c_{\text{est}}$ supplies the mode and a Gaster-type seed $\alpha_i \approx -\alpha_r c_i/c_r$ [8], and Newton iteration on the dispersion residual $F(\alpha) = \alpha c(\alpha) - \omega = 0$ converges to the spatial eigenvalue. The temporal solver’s no-guess robustness thus bootstraps the spatial solver’s physically relevant answer; both spectral spatial routes return the same σ (*Numerical Methods*, Section 4).

	Temporal (real α , complex c)	Spatial (real ω , complex α)
Algebraic form	linear GEVP, Eq. (3)	quadratic QEP, Eq. (4)
Needs a guess?	no (full spectrum by QZ)	yes (shift target or temporal seed)
Growth rate	$\omega_i = \alpha c_i$	$\sigma = -\alpha_i$
Used in this repository for	mode discovery and surveys; growth maps; temporal neutral curves; Mack Table 10.1 and Fig. 10.6 checks; Özgen Fig. 3 lobes	fixed-frequency growth curves; spatial neutral branches; N -factors; Malik Case 6 anchor; Mach 6 production case; Mach 5.35 collaborator benchmark
Typical entry points	<code>temporal_mode</code> , <code>temporal_growth_curve</code> , <code>trace_temporal_neutral_curve</code>	<code>spatial_mode</code> , <code>spatial_growth_curve</code> , <code>trace_spatial_neutral_curve</code>

Table 2: Division of labour between the two formulations. The temporal problem is the no-guess survey instrument; the spatial problem produces what transition prediction consumes.

5 Stage 3, second decision: which eigenvalue engine

Within a formulation, `pyMack` keeps several numerical routes to the same eigenvalue, deliberately: independent discretisations that agree on a mode constitute a verification statement no single method can make [2]. The four engines and their roles:

Engine 1: temporal spectral (global). Chebyshev collocation [4, 9] plus dense QZ on Eq. (3) (`temporal_solver.py` in Özgen’s temperature-form energy arrangement, `solver.py` in Mack’s enthalpy form; the two arrangements bracket the closure choice and agree on the physical mode to discretisation accuracy). Chosen whenever the spectrum is unknown: surveys, first looks at a new condition, neutral-curve sweeps.

Engine 2: spatial spectral (global, targeted). The companion linearisation of Eq. (4) with shift–invert selection near α_0 , or the Newton route from a temporal seed (`solver.py`). Chosen for every product that needs σ at fixed frequency, which is all of stage 6.

Engine 3: shooting (local). The classical initial-value route [1, 3]: the disturbance equations recast as a first-order system $dY/dy = A(y)Y$ in a six-component state, the three freestream-decaying solutions of the constant-coefficient limit A_∞ marched from y_{max} to the wall by RK4 with QR re-orthonormalisation after every step [10], and the eigenvalue located by driving the smallest singular value of the 3×3 wall matrix to zero,

$$\sigma_{\min}(M(c)) \rightarrow 0 \iff c \text{ is an eigenvalue,} \quad (5)$$

by Nelder–Mead search [11] (*Shooting Solver* document; `temporal_shooting.py`, `mack_shooting.py`). The shooting engine returns *one* eigenvalue and needs a good guess, so it is never the survey tool. It is used in exactly three situations:

1. **Independent verification.** Agreement between spectral and shooting eigenvalues at the same conditions is the repository’s strongest internal check that a mode is physics rather than discretisation artifact (`validation/test_temporal_shooting_cross_check.py`).
2. **Refinement.** A spectral eigenvalue accurate to discretisation error is polished to tight tolerance by the local search seeded with the spectral value.
3. **Regimes where the reduced EVP misses a branch.** For the low-/mid-Mach oblique first mode, the exact first-order shooting system (Mack’s Appendix A operator, extended to the sixth- and eighth-order oblique systems) resolves the Table 10.1 growth rates to 0.07–0.91%, while the reduced temporal EVP returns a different, unphysical branch family; the shooting path is authoritative there (*Validation*; the Mack Table 10.1 machinery in `mack_table_10.1.py` and `analysis.py:temporal_growth_scan_3d_shooting`).

Engine 4: the dense spatial backend. `pymack/dense.py` is a compact, independently written spatial QEP path with its own Lees–Dorodnitsyn base flow and most-amplified-seed curve tracking. It exists as an *independent reference implementation* for the Mach 6 second-mode production work: the cross-method consistency gate (`validation/test_spatial_cross_method_consistency.py`) holds the dense backend and the main `solve_spatial` family to $|\Delta\alpha_r| \leq 2 \times 10^{-4}$, $|\Delta\sigma| \leq 10^{-4}$ at the canonical Mach 6 point. The production Mach 6 runner uses it as its default backend (Section 7).

Stage 4, always: mode acceptance. Whatever the engine, a raw spectral solve returns $4n$ eigenvalues, most of them the finite-grid rendering of the continuous spectrum. Two tests select the discrete boundary-layer mode (*Numerical Methods*, Section 5): the eigenfunction must decay toward the freestream (edge-to-peak amplitude below 0.1), and the eigenvalue must be stationary under a change of domain height. The facade applies the decay test always and the stationarity test automatically when no user guess anchors the branch (`api.py`). The worked example of *Numerical Methods* Section 6 shows why the tests are not optional: at $M_e = 6$, $R = 5500$ the largest-growth raw eigenvalue is spurious, and the physical Mack mode is the *second* entry of the sorted list.

6 Stages 5–6: from eigenvalues to deliverables

Every engineering product is an aggregation of accepted stage-4 eigenvalues.

Growth curves. At fixed R : sweep α and record $\omega_i = \alpha c_i$ (temporal, `temporal_growth_curve`) or sweep ω and record $\sigma = -\alpha_i$ (spatial, `spatial_growth_curve`). These are the per-station instability portraits and the unit of comparison for most literature checks.

Neutral curves. The neutral curve is the zero-growth contour of a growth field over a two-parameter sweep: $\omega_i(R, \alpha) = 0$ or $\sigma(R, \omega) = 0$. It is traced by sweeping the grid, applying stage 4 at every node, and extracting the zero contour, with secant or Brent refinement of the crossings and continuation for branches that approach the continuous spectrum (`trace_temporal_neutral_curve`, `trace_spatial_neutral_curve`, `analysis.py`). Each fixed R inside the unstable pocket yields two crossings, the lower (branch I) and upper (branch II) neutral points; the critical Reynolds number is the nose where the two branches meet (`critical_reynolds_by_max_growth`).

***N*-factor.** The transition estimate follows a *fixed physical frequency* $F = \omega_L/R$ downstream: the spatial problem is solved at successive stations R , and the accumulated amplification is

$$N(R) = \int_{R_0}^R \max(\sigma_L, 0) dR', \quad \frac{A(R)}{A(R_0)} = e^{N(R)}, \quad (6)$$

integrated by the trapezoid rule from the lower neutral point R_0 of that frequency (`integrate_n_factor`, `analysis.py`). Transition is empirically correlated with $N \approx 9$ –11 in flight and $N \approx 4$ –6 in noisy conventional tunnels, the e^N method of Smith & Gamberoni and van Ingen [12, 13]. The envelope over frequencies of $N(R)$ curves is the transition-prediction product. Note the chain of dependencies: the N -factor needs spatial growth rates at fixed F (stage 5, spatial formulation, engines 2 or 4), which need accepted eigenvalues (stage 4), which need the operator (stage 3), which needs the profile (stage 2). A single wrong stage poisons everything downstream, which is why the validation pyramid of Section 8 tests the stages independently.

Dimensionalisation. The entire eigenvalue pipeline is nondimensional. Physical units enter only at the very end, through a `DimensionalEdgeState` (U_e, ν_e, T_e) and the converters in `scales.py`: $F \rightarrow f$ [kHz], $R \rightarrow x$ [mm], $\sigma_L \rightarrow \sigma$ [1/m]. The conversions apply the same L^* definition the solver used, so no new physics and no new approximation enters at this stage.

Geometry: the sharp cone. For a sharp cone at zero incidence, the Mangler transformation maps the cone boundary layer at surface distance s onto the flat-plate problem at $x_{\text{eq}} = s/3$, independent of half-angle [1]; `pymack/cone.py` is pure station bookkeeping ($R_{\text{eq}} = R_s/\sqrt{3}$, $N_{\text{cone}} = 3N_{\text{plate}}$ over the same R_{eq} window) and the solvers themselves are untouched. The cone path therefore reuses stages 2–6 verbatim, changing only the interpretation of the abscissa.

7 The canonical Mach 6 case, stage by stage

The production case behind the README figures (`scripts/run_mach6_spatial_neutral_case.py`; conditions $M_e = 6$, air, Sutherland, $T_e = 300$ K, $T_w/T_e = 5.88$, isothermal wall) exercises the full map:

1. **Stage 2.** One mean-flow solve at the stated wall and transport conditions.
2. **Stage 3, formulation.** Spatial: the deliverable is a neutral curve in the (R, F) plane and N -factors, both fixed-frequency products.
3. **Stage 3, engine.** The dense spatial backend with continuation (`pymack_dense` + `pymack_continuation`), cross-checked against the main family by the consistency gate.
4. **Stages 4–5.** One fixed-frequency sweep per $F \in [6.0 \times 10^{-5}, 2.30 \times 10^{-4}]$ over $R \in [200, 3300]$, second-mode branch selected by the phase-speed window $0.90 \leq c \leq 0.97$; lower and upper neutral crossings extracted per frequency (policy: single sweep, no stitching, no smoothing, recorded in the run manifest).
5. **Stage 6.** $N(R)$ and e^N per frequency by Eq. (6); dimensional overlays via `DimensionalEdgeState`.
Expected anchor: at $F = 6.0 \times 10^{-5}$, $R_{\text{lower}} \approx 1543$, $R_{\text{upper}} \approx 3041$ (`docs/MACH6_SPATIAL_NEUTRAL_WORKFLOW.m`).

Run `--geometry cone --cone-half-angle-deg 7` and the identical chain serves the cone through the Mangler bookkeeping of Section 6.

8 Where validation attaches to the map

The layered validation strategy (`docs/VALIDATION_STRATEGY.md`; *Validation*) verifies the stages independently, so a failure localises to a stage instead of surfacing as an unexplained error in a final product. Table 3 lists, for each layer, the solver route that produced the compared quantity, so every verdict can be traced back to a specific engine of Section 5:

Layer	Stage verified: solver route (what it computed)	Benchmark	Status
0	spectral machinery: D_1, D_2 and the domain map applied to analytic functions; no eigenvalue solver	analytic derivatives	machine precision
1	stage 2: mean-flow similarity BVP (<code>baseflow.py</code> ; <code>solve_bvp</code> [14]), thicknesses by quadrature	Mack Table 11.1 thicknesses [1]	$< 0.5\%$
2	stages 3–4: Orr–Sommerfeld temporal GEVP by QZ (<code>solve_temporal_os</code> , <code>solve_poiseuille_temporal</code>); least-stable eigenvalue	Orszag Poiseuille eigenvalue [4]	5+ significant figures
3	stages 3–4: exact oblique first-order shooting, 6×6 and 8×8 systems (engine 3; <code>evaluate_table_10.1_exact_shooting</code>); temporal growth rates ω_i	Mack Table 10.1 [1]	0.07–0.91%
4a	stage 3, spatial: five routes at one point, dense companion QEP (<code>dense.solve_mack_branch</code>) vs shift-invert QEP (<code>solve_spatial</code>) vs Newton (<code>solve_spatial_newton</code>) vs Gaster-seeded pipeline (<code>solve_spatial_from_temporal</code>) vs Muller (<code>solve_spatial_muller</code>); complex α	internal cross-method redundancy	independent operators $\leq 2 \times 10^{-4}$
4b	stage 3, spatial: shift-invert QEP (<code>solve_spatial</code>); complex α at Malik’s printed conditions	Malik (1990) Table IX Case 6 [2]	α to $\sim 5 \times 10^{-6}$
5	stages 2–6 end to end: production fixed-frequency spatial sweep (<code>compute_spatial_fixed_frequency_curves.py</code>), neutral branches dimensionalised via <code>scales.py</code> ; committed artifact with run manifest	independent-code Mach 5.35 N_2 neutral curve	upper branch MAE 3.2 mm
6	overlays: Özgen lobes from the temperature-form temporal GEVP (<code>solve_temporal_2d</code>); Mack Fig. 10.3 from the exact-shooting scan (engine 3)	Özgen Fig. 3, Mack Fig. 10.3 [3, 1]	demonstrations, not gates

Table 3: The validation pyramid mapped onto the pipeline stages, with the solver route behind each verdict. Tables and independent codes gate; figures illustrate.

The verification audit, case by case. Beyond the CI gates, the standalone audit (`verification/SUCCESS_MATRIX.md`) runs published benchmark conditions and records a verdict per case. Table 4

states which solver produced each compared quantity; the pattern is the division of labour of Sections 4–5 in action: temporal GEVP sweeps for growth-rate figures, the spatial QEP family for neutral branches, eigenvalue anchors, and N -factors, and the exact shooting engine wherever the reduced EVP’s oblique first-mode branch selection is not trusted.

Case	Solver route	Quantity compared	Verdict
Mack Fig. 10.6, $M=4.5$ –10 [1]	temporal GEVP sweep over α (<code>solve_temporal_compressible</code>)	second-mode max ω_i vs M	1–7%
Ma & Zhong (2003), $M=4.5$ [15]	spatial QEP fixed-frequency sweeps (<code>solve_spatial</code>), $\sigma = 0$ crossings	second-mode neutral branches I/II	~3%
Balakumar & Malik (1992) [16]	full companion-QEP spectrum (<code>solve_spatial_full_spectrum</code>)	discrete eigenvalue + discrete/continuous branch topology	α_r exact; α_i ~10%
Egorov et al. (2006), $M=6$	spatial QEP growth curves (<code>solve_spatial</code>)	most-amplified frequency vs DNS-LST	~7%
Sivasubramanian & Fasel (2015), $M=6$ cone	spatial QEP (<code>solve_spatial</code>) + Mangler bookkeeping (<code>cone.py</code>) + trapezoid N ; Taylor–Maccoll edge state	sharp-cone N -factor	$N \approx 7.1$ vs ~7–8
Malik (1990) anchors [2]	Cases 3–5: temporal GEVP (<code>solve_temporal_compressible</code> , <code>_3d</code>); Case 6, Table X: spatial QEP (<code>solve_spatial</code>); Fig. 4: temporal eigenfunction	eigenvalue anchors + second-mode eigenfunction	anchors within literature spread
Mack Fig. 10.1, $M=2.2$ [1]	temporal GEVP neutral tracing (<code>solve_temporal_compressible</code>)	first-mode neutral loop	documented weak spot
Mack Fig. 10.3 [1]	exact 3D shooting continuation from an anchor root (<code>temporal_growth_scan_3d_shooting_from_anchor</code>)	oblique first-mode $\omega_i(R)$	open ($M \approx 2.2$)
Mack Fig. 10.4 [1]	reduced 3D temporal GEVP (<code>solve_temporal_compressible_3d</code>)	high- M oblique first mode	under-amplified
Özgen Fig. 3, $M=2$ –8 [3]	temperature-form temporal GEVP (<code>solve_temporal_2d</code>) + discrete-mode acceptance + branch continuation	first/second mode growth lobes	mixed (second mode agrees; first mode under-amplified)

Table 4: The verification-audit cases with the solver route behind each verdict. Verdicts paraphrase `verification/SUCCESS_MATRIX.md`, which remains the single source of truth; each case folder’s `verdict.json` records metrics and provenance.

A confidence note: the second (Mack) mode, the design target, is validated across Mach 4.5–10 against Mack, Ma & Zhong, Balakumar & Malik, Egorov et al., and a sharp-cone N -factor comparison [1, 15, 16]; the first mode at low-to-moderate Mach is systematically under-amplified in the reduced EVP path and is the documented weak spot, partially recovered by the exact shooting engine (Section 5, item 3).

References

- [1] L. M. Mack, “Boundary-layer linear stability theory,” AGARD Report No. 709, *Special Course on Stability and Transition of Laminar Flow*, 1984.
- [2] M. R. Malik, “Numerical methods for hypersonic boundary layer stability,” *Journal of Computational Physics*, Vol. 86, No. 2, 1990, pp. 376–413.
- [3] S. Özgen and S. A. Kırcah, “Linear stability analysis in compressible, flat-plate boundary layers,” *Theoretical and Computational Fluid Dynamics*, Vol. 22, 2008, pp. 1–20.
- [4] S. A. Orszag, “Accurate solution of the Orr–Sommerfeld stability equation,” *Journal of Fluid Mechanics*, Vol. 50, 1971, pp. 689–703.
- [5] P. J. Schmid and D. S. Henningson, *Stability and Transition in Shear Flows*, Springer, 2001.
- [6] C. B. Moler and G. W. Stewart, “An algorithm for generalized matrix eigenvalue problems,” *SIAM Journal on Numerical Analysis*, Vol. 10, 1973, pp. 241–256.
- [7] F. Tisseur and K. Meerbergen, “The quadratic eigenvalue problem,” *SIAM Review*, Vol. 43, No. 2, 2001, pp. 235–286.
- [8] M. Gaster, “A note on the relation between temporally-increasing and spatially-increasing disturbances in hydrodynamic stability,” *Journal of Fluid Mechanics*, Vol. 14, 1962, pp. 222–224.
- [9] L. N. Trefethen, *Spectral Methods in MATLAB*, SIAM, 2000.
- [10] S. D. Conte, “The numerical solution of linear boundary value problems,” *SIAM Review*, Vol. 8, No. 3, 1966, pp. 309–321.
- [11] J. A. Nelder and R. Mead, “A simplex method for function minimization,” *The Computer Journal*, Vol. 7, No. 4, 1965, pp. 308–313.
- [12] A. M. O. Smith and N. Gamberoni, “Transition, pressure gradient and stability theory,” Douglas Aircraft Co. Report ES-26388, 1956.
- [13] J. L. van Ingen, “A suggested semi-empirical method for the calculation of the boundary layer transition region,” Delft University of Technology, Report VTH-74, 1956.
- [14] J. Kierzenka and L. F. Shampine, “A BVP solver based on residual control and the MATLAB PSE,” *ACM Transactions on Mathematical Software*, Vol. 27, No. 3, 2001, pp. 299–316.
- [15] Y. Ma and X. Zhong, “Receptivity of a supersonic boundary layer over a flat plate. Part 1. Wave structures and interactions,” *Journal of Fluid Mechanics*, Vol. 488, 2003, pp. 31–78.
- [16] P. Balakumar and M. R. Malik, “Discrete modes and continuous spectra in supersonic boundary layers,” *Journal of Fluid Mechanics*, Vol. 239, 1992, pp. 631–656.